

KUBERNATES_TASK_2

summary + importance of all tasks (no steps)

1. Simple Pod YAML

Summary: Created a basic nginx pod using YAML.

Importance: Teaches Pod structure and basic deployment. Pod is the smallest unit in Kubernetes.

2. Environment Variables in Pod

Summary: Added env variables like myname and City.

Importance: Used for app configuration (DB details, API keys, environment configs) without changing code.

3. Args / Command in Pod

Summary: Passed args like sleep 50.

Importance: Helps override container startup behavior, useful for debugging and running custom commands.

4. Pod with Two Containers

Summary: Ran two containers inside one Pod.

Importance: Sidecar concept. Both containers share same network/IP and can communicate via localhost.

5. Init Container

Summary: Init container runs before main container.

Importance: Used for pre-setup tasks like waiting for DB, downloading configs, setting permissions.

6. Dry-run YAML Generation

Summary: Generated YAML using dry-run and modified it.

Importance: Quick YAML creation, reduces manual mistakes, useful in real-time DevOps work.

7. kubectl create vs apply

Summary: Used create first time, apply for updates.

Importance: create = only new resource, apply = create + update (used in production).

8. kubectl edit pod

Summary: Edited running pod config.

Importance: Emergency troubleshooting method (but not best practice for production).

9. Expose Pod with Service

Summary: Created ClusterIP service for pod access.

Importance: Pod IP changes, Service gives stable DNS/IP and load balancing inside cluster.

10. Resource Requests & Limits

Summary: Set CPU/memory requests and limits.

Importance: Prevents resource overuse, improves scheduling, must-have for production clusters.

Overall Importance

These tasks cover real Kubernetes fundamentals:

Pod deployment

configuration

multi-container pod concept

init containers

kubectl operations

networking via services

production resource management

TASK k8s-02 tasks.

1. **Create a Simple Pod Using YAML Task:** Write a YAML file to create a Pod named `firstpod` with an `nginx` container. Verify the Pod creation using `kubectl get pods` and check the logs of the container using `kubectl logs firstpod`.
2. **Set Environment Variables in a Pod Task:** Modify the YAML file to include environment variables `myname: sabair` and `City: Hyderabad`. Deploy the Pod and use `kubectl exec <pod_name> -- env` to check if the environment variables are set properly.
3. **Deploy a Pod with Commands (Args) in YAML Task:** Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use `kubectl describe pod` to verify the args are correctly passed to the container.
4. **Create a Pod with Two Containers Task:** Create a YAML file to define a Pod with two `nginx` containers inside. Use `kubectl exec` to access both containers and verify that both containers can communicate through the same network (e.g., using `telnet` between them).
5. **Set Up an Init Container in a Pod Task:** Modify the YAML to include an `init` container that sleeps for 30 seconds before the main containers start. Verify the `init` container's execution using `kubectl describe pod` and check the logs to confirm its completion.

6. Run a Dry Run Command to Generate YAML Task: Use the `kubectl run nginx --image=nginx --dry-run=client -o yaml` command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.
7. Use `kubectl apply` vs `kubectl create` Task: Create a YAML file to define a Pod. First, deploy it using `kubectl create -f <file_name>.yaml` and then modify the YAML (e.g., change the image version). Use `kubectl apply` to redeploy and verify the difference between both commands.
8. Edit an Existing Pod Configuration Task: Use `kubectl edit pod <pod_name>` to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using `kubectl exec`.
9. Expose a Pod Using a Service Task: Create a YAML file to expose your firstpod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using `kubectl get svc`.
10. Pod with Resource Limits and Requests Task: Add resource requests and limits to the containers in your YAML file. Specify CPU and memory requests/limits for both containers and deploy the Pod. Use `kubectl describe pod` to verify if the resource configurations are correctly applied.

Ok bro 🤓 let's do **step by step** like a lab.

TASK 1: Create a Simple Pod Using YAML

Step 1: Create YAML file

Create a file:

`vi firstpod.yaml`

Paste this YAML:

`apiVersion: v1`

`kind: Pod`

`metadata:`

name: firstpod

spec:

containers:

- name: nginx

image: nginx

Save and exit.

```
root@master:~# vi firstpod.yaml
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: nginx
    image: nginx
```

Step 2: Create Pod

kubectl apply -f firstpod.yaml

```
root@master:~# kubectl apply -f firstpod.yaml
pod/firstpod created
root@master:~#
```

Step 3: Verify Pod

kubectl get pods

Expected output:

firstpod Running

```
root@master:~# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
firstpod                            1/1     Running   0           111s
nginx-deploy-6f47956ff4-5cc19       1/1     Running   1 (19m ago)  18h
nginx-deploy-6f47956ff4-d4c75       1/1     Running   1 (19m ago)  18h
root@master:~#
```

Step 4: Check Pod logs

kubectl logs firstpod

(nginx logs may show nothing unless traffic comes, that's normal)

```

root@master:~# kubectl logs firstpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/17 09:56:09 [notice] 1#1: using the "epoll" event method
2026/02/17 09:56:09 [notice] 1#1: nginx/1.29.5
2026/02/17 09:56:09 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/17 09:56:09 [notice] 1#1: OS: Linux 6.14.0-1018-aws
2026/02/17 09:56:09 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/17 09:56:09 [notice] 1#1: start worker processes
2026/02/17 09:56:09 [notice] 1#1: start worker process 30
2026/02/17 09:56:09 [notice] 1#1: start worker process 31
root@master:~#

```

✓ Task 1 completed.

TASK 2: Set Environment Variables in Pod

Step 1: Edit YAML

Open file:

vi firstpod.yaml

Modify like this:

apiVersion: v1

kind: Pod

metadata:

name: firstpod

spec:

containers:

- name: nginx

image: nginx

env:

- name: myname

value: sabair

- name: City

value: Hyderabad

```

root@master:~# vi firstpod.yaml
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: nginx
    image: nginx
    env:
    - name: myname
      value: sabair
    - name: City
      value: Hyderabad

```

Step 2: Reapply YAML

⚠ Pod already exists, so delete and recreate:

kubectl delete pod firstpod

kubectl apply -f firstpod.yaml

```

root@master:~# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
firstpod            1/1     Running   0           11s
nginx-deploy-6f47956ff4-5cc19  1/1     Running   1 (32m ago)  18h
nginx-deploy-6f47956ff4-d4c75  1/1     Running   1 (32m ago)  18h
root@master:~# kubectl get pod firstpod -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE     NOMINATED NODE   READINESS GATES
firstpod            1/1     Running   0           3m45s  10.10.226.72    worker-1 <none>          <none>
root@master:~# kubectl delete pod firstpod
pod "firstpod" deleted from default namespace
root@master:~# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
nginx-deploy-6f47956ff4-5cc19  1/1     Running   1 (35m ago)  18h
nginx-deploy-6f47956ff4-d4c75  1/1     Running   1 (35m ago)  18h
root@master:~#

```

Step 3: Verify env variables

kubectl exec firstpod -- env

```

root@master:~# kubectl exec firstpod -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=firstpod
NGINX_VERSION=1.29.5
NJS_VERSION=0.9.5
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
myname=sabair
City=Hyderabad
KUBERNETES_PORT_443_TCP_PORT=443
NGINX_DEPLOY_SERVICE_PORT=80
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
NGINX_DEPLOY_PORT_80_TCP_ADDR=10.104.55.240
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
NGINX_DEPLOY_SERVICE_HOST=10.104.55.240
NGINX_DEPLOY_PORT_80_TCP=tcp://10.104.55.240:80
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
NGINX_DEPLOY_PORT=tcp://10.104.55.240:80
NGINX_DEPLOY_PORT_80_TCP_PROTO=tcp
NGINX_DEPLOY_PORT_80_TCP_PORT=80
KUBERNETES_SERVICE_HOST=10.96.0.1
HOME=/root

```

To filter only your variables:

kubectl exec firstpod -- env | grep -i myname

```
kubectl exec firstpod -- env | grep -i city
```

Expected:

myname=sabair

City=Hyderabad

```
HOME=/root
root@master:~# kubectl exec firstpod -- env | grep -i myname
myname=sabair
root@master:~# kubectl exec firstpod -- env | grep -i city
City=Hyderabad
root@master:~#
```

✓ Task 2 completed.

TASK 3: Pod with args (sleep 50 seconds)

Step 1: Update YAML

Edit again:

```
vi firstpod.yaml
```

Replace container part like this:

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: firstpod
```

```
spec:
```

```
  containers:
```

```
  - name: nginx
```

```
    image: nginx
```

```
    args: ["sleep", "50"]
```

```
root@master:~# vi firstpod.yaml
root@master:~# cat firstpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
  - name: nginx
    image: nginx
    args: ["sleep", "50"]
```

⚠ NOTE: nginx image normally runs nginx process.
If you set args sleep, nginx won't run. It will just sleep.

Step 2: Recreate Pod

kubectl delete pod firstpod

kubectl apply -f firstpod.yaml

```
pod/firstpod created
root@master:~# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
firstpod                            1/1     Running   0           51s
nginx-deploy-6f47956ff4-5cc19       1/1     Running   1 (52m ago)  19h
nginx-deploy-6f47956ff4-d4c75       1/1     Running   1 (52m ago)  19h
root@master:~#
```

Step 3: Verify args

kubectl describe pod firstpod

Look inside:

Args:

sleep

50

```

root@master:~# kubectl describe pod firstpod
Name:          firstpod
Namespace:     default
Priority:       0
Service Account: default
Node:          worker-1/172.31.46.47
Start Time:    Tue, 17 Feb 2026 10:31:00 +0000
Labels:        <none>
Annotations:   cn1.projectcalico.org/containerID: 5b66913f6edb1d790cd7b0f773edda18575ccb7b7e3a15230ab143e44802306b
               cn1.projectcalico.org/podIP: 10.10.226.74/32
               cn1.projectcalico.org/podIPs: 10.10.226.74/32
Status:        Running
IP:            10.10.226.74
IPs:           10.10.226.74
Containers:
  nginx:
    Container ID:   containerd://02f1b3591c27d00c3aa692a0200b205d44a1ac5e5088abe5ebb8b46504a583e8
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
    Port:           <none>
    Host Port:      <none>
    Args:           sleep
                   50
    State:          Running
      Started:      Tue, 17 Feb 2026 10:34:11 +0000
    Last State:     Terminated
      Reason:       Completed
    Exit Code:      0
      Started:      Tue, 17 Feb 2026 10:32:55 +0000
      Finished:     Tue, 17 Feb 2026 10:33:45 +0000
    Ready:          True
    Restart Count:  3
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-kq152 (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True

```

✓ Task 3 completed.

TASK 4: Pod with Two Containers

Step 1: Create YAML

Create file:

vi twocontainerpod.yaml

Paste:

apiVersion: v1

kind: Pod

metadata:

name: twopod

spec:

containers:

- name: nginx1

image: nginx

- name: nginx2

image: nginx

```
root@master:~# vi twocontainerpod.yaml
root@master:~# cat twocontainerpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: twopod
spec:
  containers:
  - name: nginx1
    image: nginx
  - name: nginx2
    image: nginx
root@master:~#
```

Step 2: Apply

kubectl apply -f twocontainerpod.yaml

```
image: nginx
root@master:~# kubectl apply -f twocontainerpod.yaml
pod/twopod created
root@master:~# kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
firstpod                            0/1     CrashLoopBackoff    4 (69s ago) 6m52s
nginx-deploy-6f47956ff4-5cc19       1/1     Running             1 (58m ago) 19h
nginx-deploy-6f47956ff4-d4c75       1/1     Running             1 (58m ago) 19h
twopod                              1/2     Error                1 (11s ago) 15s
root@master:~#
```

Step 3: Verify

kubectl get pods

Step 4: Exec into container 1

kubectl exec -it twopod -c nginx1 -- bash

```
twopod
root@master:~# kubectl exec -it twopod -c nginx1 -- bash
root@twopod:/#
```

Step 5: From nginx1 ping nginx2?

Inside same pod, both share same network → **same localhost**.

So test like:

curl localhost

If curl not installed:

apt update

apt install curl -y

curl localhost

Now exit.

```
kind: Pod
metadata:
  name: twopod
spec:
  containers:
  - name: nginx1
    image: nginx
  - name: nginx2
    image: nginx

root@master:~# kubectl apply -f twocontainerpod.yaml
pod/twopod created
root@master:~# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
firstpod                           0/1     CrashLoopBackOff   4 (69s ago)    6m52s
nginx-deploy-6f47956ff4-5cc19      1/1     Running    1 (58m ago)   19h
nginx-deploy-6f47956ff4-d4c75      1/1     Running    1 (58m ago)   19h
twopod                              1/2     Error      1 (11s ago)   15s
root@master:~# kubectl exec -it twopod -c nginx1 -- bash
root@twopod:/# curl localhost
<!DOCTYPE html>
<html>
<head>
<title>welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Step 6: Exec into nginx2

kubectl exec -it twopod -c nginx2 -- bash

curl localhost

✅ Both containers share same IP, so localhost works.

```

<head>
<title>welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@twopod:/# exit
exit
root@master:~# kubectl exec -it twopod -c nginx2 -- bash
root@twopod:/# curl localhost
<!DOCTYPE html>
<html>
<head>
<title>welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@twopod:/# |

```



Task 4 completed.

TASK 5: Init Container (sleep 30 seconds)

Step 1: Create YAML

vi initpod.yaml

Paste:

cat > initpod.yaml <<EOF

apiVersion: v1

kind: Pod

metadata:

name: initpod

spec:

initContainers:

- name: init-sleeper

image: busybox

```
command: ["sh", "-c", "sleep 30"]
```

containers:

- name: nginx

image: nginx

```
root@master:~# vi initpod.yaml
root@master:~# cat initpod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: initpod
spec:
  initContainers:
  - name: init-sleeper
    image: busybox
    command: ["sh", "-c", "sleep 30"]
  containers:
  - name: nginx
    image: nginx
root@master:~#
```

Step 2: Apply

```
kubectl apply -f initpod.yaml
```

```
- name: nginx
  image: nginx
root@master:~# kubectl apply -f initpod.yaml
pod/initpod created
root@master:~#
```

Step 3: Verify init container running

```
kubectl describe pod initpod
```

```
root@master:~# kubectl describe pod initpod
Name:          initpod
Namespace:     default
Priority:       0
Service Account: default
Node:          worker-1/172.31.46.47
Start Time:    Tue, 17 Feb 2026 11:52:10 +0000
Labels:        <none>
Annotations:   cni.projectcalico.org/containerID: 763c30f217f62cccb194f5d89c291eb23924c9f7f57c59b4ca9b80c315f250ea
               cni.projectcalico.org/podIP: 10.10.226.75/32
               cni.projectcalico.org/podIPs: 10.10.226.75/32
Status:        Running
IP:            10.10.226.75
Init Containers:
  init-sleeper:
    Container ID:  containerd://79511cae882f5d5ea58bd995bbf583cbbb64644972b55782e0053b51b0ca674d
    Image:          busybox
    Image ID:       docker.io/library/busybox@sha256:b3255e7dfbcd10cb367af0d409747d511aeb66dfac98cf30e97e87e4207dd76f
    Port:          <none>
    Host Port:     <none>
    Command:
      sh
      -c
      sleep 30
    State:          Terminated
    Reason:         Completed
    Exit Code:      0
    Started:        Tue, 17 Feb 2026 11:52:12 +0000
    Finished:       Tue, 17 Feb 2026 11:52:42 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-lt4zj (ro)
Containers:
  nginx:
    Container ID:  containerd://20fa719f08a0ae9caa7c270185b189057c8bd0ef8bd4aab03fc3af8f16f692ad
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
```

You will see:

- Init Containers status
- Main container starts after init completes

Step 4: Check init container logs

`kubectl logs initpod -c init-sleeper`

✅ Task 5 completed.

TASK 6: Dry Run to Generate YAML

Step 1: Generate YAML

`kubectl run nginx --image=nginx --dry-run=client -o yaml > dryrunpod.yaml`

```
root@master:~# kubectl pods
error: unknown command "pods" for "kubectl"

Did you mean this?
  logs
root@master:~# kubectl get pods
NAME                READY   STATUS              RESTARTS   AGE
firstpod            0/1     CrashLoopBackOff   18 (35s ago)  83m
initpod             1/1     Running             0           2m20s
nginx-deploy-6f47956ff4-5cc19  1/1     Running             1 (135m ago)  20h
nginx-deploy-6f47956ff4-d4c75  1/1     Running             1 (135m ago)  20h
twopod              2/2     Running             0           65m
root@master:~# kubectl run nginx --image=nginx --dry-run=client -o yaml > dryrunpod.yaml
root@master:~# ls
custom-resources.yaml  dryrunpod.yaml  firstpod.yaml  initpod.yaml  snap  twocontainerpod.yaml
root@master:~# |
```

Step 2: Check file

```
cat dryrunpod.yaml
```

Step 3: Modify YAML (add label + env)

Edit file:

```
vi dryrunpod.yaml
```

Add:

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  labels:
```

```
    run: nginx
```

```
  name: nginx
```

```
spec:
```

```
  containers:
```

```
    - image: nginx
```

```
      name: nginx
```

```
      resources: {}
```

```
  dnsPolicy: ClusterFirst
```

```
  restartPolicy: Always
```

```
status: {}
```

```
custom-resources.yaml dryrunpod.yaml firstpod.yaml initpod.yaml snap twocontainerpod.yaml
root@master:~# cat dryrunpod.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
root@master:~# |
```

Step 4: Apply

kubectl apply -f dryrunpod.yaml

Step 5: Verify

```
root@master:~# kubectl run nginx --image=nginx --dry-run=client -o yaml > dryrunpod.yaml
root@master:~# ls
custom-resources.yaml dryrunpod.yaml firstpod.yaml initpod.yaml snap twocontainerpod.yaml
root@master:~# cat dryrunpod.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
root@master:~# kubectl apply -f dryrunpod.yaml
pod/nginx created
root@master:~# |
```

TASK 7: kubectl apply vs create

Step 1: Create YAML

vi applycreate.yaml

Paste:

apiVersion: v1

kind: Pod

metadata:

name: applypod

spec:

containers:

- name: nginx

image: nginx:1.25

```
root@master:~# vi applycreate.yaml
root@master:~# cat applycreate.yaml
apiVersion: v1
kind: Pod
metadata:
  name: applypod
spec:
  containers:
  - name: nginx
    image: nginx:1.25
```

Step 2: Deploy using create

kubectl create -f applycreate.yaml

```
image: nginx:1.25
root@master:~# kubectl create -f applycreate.yaml
pod/applypod created
root@master:~# |
```

Step 3: Modify YAML

Change image version:

image: nginx:latest

```
root@master:~# vi applycreate.yaml
root@master:~# cat applycreate.yaml
apiVersion: v1
kind: Pod
metadata:
  name: applypod
spec:
  containers:
  - name: nginx
    image: latest
root@master:~# |
```

Step 4: Deploy using create again

kubectl create -f applycreate.yaml

```
root@master:~# vi applycreate.yaml
root@master:~# kubectl create -f applycreate.yaml
Error from server (AlreadyExists): error when creating "applycreate.yaml": pods "applypod" already exists
root@master:~# |
```

You will get error:

already exists

Step 5: Deploy using apply

kubectl apply -f applycreate.yaml

```
root@master:~# kubectl apply -f applycreate.yaml
warning: resource pods/applypod is missing the kubect1.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be used on resources created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.
pod/applypod configured
root@master:~#
```

Step 6: Verify image updated

kubectl describe pod applypod | grep -i image

```
root@master:~# kubectl describe pod applypod | grep -i image
Image: latest
Image ID: docker.io/library/nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c
Normal Pulling 5m45s kubelet Pulling image "nginx:1.25"
Normal Pulled 5m41s kubelet Successfully pulled image "nginx:1.25" in 3.907s (3.907s including waiting). Image
size: 71005258 bytes.
Normal BackOff 2m5s (x2 over 2m35s) kubelet Back-off pulling image "latest"
Warning Failed 2m5s (x2 over 2m35s) kubelet Error: ImagePullBackOff
Normal Pulling 68s (x4 over 2m35s) kubelet Pulling image "latest"
Warning Failed 68s (x4 over 2m35s) kubelet Failed to pull image "latest": failed to pull and unpack image "docker.io/library/
latest:latest": failed to resolve reference "docker.io/library/latest:latest": pull access denied, repository does not exist or may require autho
rization: server message: insufficient_scope: authorization failed
Warning Failed 68s (x4 over 2m35s) kubelet Error: ErrImagePull
root@master:~#
```

✓ Difference:

- **create** = only creates once, fails if exists
- **apply** = creates + updates existing object

✓ Task 7 completed.

TASK 8: Edit an Existing Pod

Step 1: Edit Pod

kubectl edit pod applypod

```
root@master: ~  
# Please edit the object below. Lines beginning with a '#' will be ignored,  
# and an empty file will abort the edit. If an error occurs while saving this file will be  
# reopened with the relevant failures.  
#  
apiVersion: v1  
kind: Pod  
metadata:  
  annotations:  
    cnf.projectcalico.org/containerID: ca0ac79d978d5d3e447875243588fce51fd21a0554061612036cc0b71ef71afa  
    cnf.projectcalico.org/podIP: 10.10.133.202/32  
    cnf.projectcalico.org/podIPs: 10.10.133.202/32  
    kubernetes.io/last-applied-configuration: |  
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"apppod","namespace":"default"},"spec":{"containers":[{"image":"lates  
t","name":"nginx"}]}}  
  creationTimestamp: "2026-02-17T12:01:38Z"  
  generation: 2  
  name: appypod  
  namespace: default  
  resourceVersion: "51602"  
  uid: e7333839-e48b-45d4-859f-fbdf51d1c793  
spec:  
  containers:  
    - image: latest  
      imagePullPolicy: IfNotPresent  
      name: nginx  
      resources: {}  
      terminationMessagePath: /dev/termination-log  
      terminationMessagePolicy: File  
      volumeMounts:  
        - mountPath: /var/run/secrets/kubernetes.io/serviceaccount  
          name: kube-api-access-k7t8s  
          readOnly: true  
  dnsPolicy: ClusterFirst  
  enableServiceLinks: true  
  nodeName: worker-2  
  preemptionPolicy: PreemptLowerPriority  
  priority: 0  
  restartPolicy: Always  
  schedulerName: default-scheduler  
  securityContext: {}  
"/tmp/kubectl-edit-3729589841.yaml" 136L, 4256B 1,1 Top
```

Try changing image:

image: nginx:1.24

Save.

```
root@master: ~  
# Please edit the object below. Lines beginning with a '#' will be ignored,  
# and an empty file will abort the edit. If an error occurs while saving this file will be  
# reopened with the relevant failures.  
#  
apiVersion: v1  
kind: Pod  
metadata:  
  annotations:  
    cnf.projectcalico.org/containerID: ca0ac79d978d5d3e447875243588fce51fd21a0554061612036cc0b71ef71afa  
    cnf.projectcalico.org/podIP: 10.10.133.202/32  
    cnf.projectcalico.org/podIPs: 10.10.133.202/32  
    kubernetes.io/last-applied-configuration: |  
      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"apppod","namespace":"default"},"spec":{"containers":[{"image":"lates  
t","name":"nginx"}]}}  
  creationTimestamp: "2026-02-17T12:01:38Z"  
  generation: 2  
  name: appypod  
  namespace: default  
  resourceVersion: "51602"  
  uid: e7333839-e48b-45d4-859f-fbdf51d1c793  
spec:  
  containers:  
    - image: 1.24  
      imagePullPolicy: IfNotPresent  
      name: nginx  
      resources: {}  
      terminationMessagePath: /dev/termination-log  
      terminationMessagePolicy: File  
      volumeMounts:  
        - mountPath: /var/run/secrets/kubernetes.io/serviceaccount  
          name: kube-api-access-k7t8s  
          readOnly: true  
  dnsPolicy: ClusterFirst  
  enableServiceLinks: true  
  nodeName: worker-2  
  preemptionPolicy: PreemptLowerPriority  
  priority: 0  
  restartPolicy: Always  
  schedulerName: default-scheduler  
  securityContext: {}  
-- INSERT -- 22,16 Top
```

Step 2: Check result

kubectl get pods

kubectl describe pod appypod | grep -i image

⚠ Note: Pods cannot always update everything directly.
If image changes, Kubernetes may recreate pod.

Step 3: Verify env

kubect exec applypod -- env

✅ Task 8 completed.

```
twopod      2/2      Running      0          84m
Image:       latest
Image ID:    docker.io/library/nginx@sha256:a484819eb60211f5299034ac80f6a681b06f89e65866ce91f356ed7c72af059c
Reason:      ImagePullBackOff
Normal      Pulling     11m         kubelet     Pulling image "nginx:1.25"
Normal      Pulled      11m         kubelet     Successfully pulled image "nginx:1.25" in 3.907s (3.907s including waiting). Image size: 71005258 bytes.
Warning     Failed      7m8s (x4 over 8m35s) kubelet     Error: ErrImagePull
Warning     Failed      5m51s (x3 over 8m35s) kubelet     Error: ImagePullBackOff
Warning     Failed      3m30s (x5 over 8m35s) kubelet     Failed to pull image "nginx:1.25": failed to pull and unpack image "docker.io/library/nginx:1.25": failed to resolve reference "docker.io/library/nginx:1.25": pull access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
Normal      Pulling     38s (x6 over 8m35s) kubelet     Pulling image "latest"
Normal      BackOff     26s (x16 over 8m35s) kubelet     Back-off pulling image "latest"
root@master:~#
```

TASK 9: Expose Pod using Service (ClusterIP)

Step 1: Create service YAML

vi service.yaml

Paste:

apiVersion: v1

kind: Service

metadata:

name: firstpod-service

spec:

type: ClusterIP

selector:

app: mynginx

ports:

- port: 80

targetPort: 80

```
root@master:~# cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: firstpod-service
spec:
  type: ClusterIP
  selector:
    app: mynginx
  ports:
  - port: 80
    targetPort: 80

root@master:~#
```

Step 2: Ensure Pod has label

Your pod should have label:

kubectl get pods --show-labels

```
root@master:~# kubectl get pods --show-labels
NAME                READY   STATUS    RESTARTS   AGE   LABELS
app1pod             0/1     ImagePullBackOff    0       14m   <none>
firstpod            0/1     CrashLoopBackOff    21 (4m9s ago)  104m   <none>
initpod             1/1     Running          0       23m   <none>
nginx                1/1     Running          0       18m   run=nginx
nginx-deploy-6f47956ff4-5cc19  1/1     Running          1 (157m ago)  20h   app=nginx-deploy,pod-template-hash=6f47956ff4
nginx-deploy-6f47956ff4-d4c75  1/1     Running          1 (157m ago)  20h   app=nginx-deploy,pod-template-hash=6f47956ff4
twopod              2/2     Running          0       86m   <none>
root@master:~#
```

If not, add label:

kubectl label pod nginx app=mynginx

Step 3: Apply service

kubectl apply -f service.yaml

```
root@master:~# kubectl apply -f service.yaml
service/firstpod-service created
root@master:~#
```

Step 4: Verify service

kubectl get svc

```
service/firstpod-service created
root@master:~# kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
firstpod-service    ClusterIP   10.110.143.156 <none>         80/TCP           57s
kubernetes          ClusterIP   10.96.0.1      <none>         443/TCP          23h
nginx-deploy        NodePort    10.104.55.240  <none>         80:32655/TCP     20h
root@master:~#
```

Step 5: Test service

kubectl exec nginx -- curl firstpod-service

```
nginx-deploy        NodePort    10.104.55.240  <none>         80:32655/TCP     20h
root@master:~# kubectl exec nginx -- curl firstpod-service
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %         %         Dload  Upload   Total   Spent    Left     Speed
  0    0    0     0     0    0     0      0      0 --:--:--  0:00:02 --:--:--    0
```

✔ Task 9 completed.

TASK 10: Pod with Resource Requests and Limits

Step 1: Create YAML

vi resourcepod.yaml

Paste:

apiVersion: v1

kind: Pod

metadata:

name: resourcepod

spec:

containers:

- name: nginx

image: nginx

resources:

requests:

cpu: "100m"

memory: "128Mi"

limits:

cpu: "200m"

memory: "256Mi"

```
root@master:~# vi resourcepod.yaml
root@master:~# cat resourcepod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: resourcepod
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"
      limits:
        cpu: "200m"
        memory: "256Mi"
```

Step 2: Apply

kubectl apply -f resourcepod.yaml

```
memory: 256Mi
root@master:~# kubectl apply -f resourcepod.yaml
pod/resourcepod created
root@master:~#
```

Step 3: Verify

kubectl describe pod resourcepod

You will see:

Requests:

cpu: 100m

memory: 128Mi

Limits:

cpu: 200m

memory: 256Mi

```

Host Port:      <none>
State:         Running
  Started:     Tue, 17 Feb 2026 12:19:32 +0000
Ready:         True
Restart Count: 0
Limits:
  cpu:         200m
  memory:      256Mi
Requests:
  cpu:         100m
  memory:      128Mi
Environment:   <none>
Mounts:
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-flfk1 (ro)
Conditions:
  Type                               Status
PodReadyToStartContainers            True
Initialized                          True
Ready                                True
ContainersReady                      True
PodScheduled                         True
Volumes:
  kube-api-access-flfk1:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt
    Optional:      false
    DownwardAPI:   true
QoS Class:           Burstable
Node-Selectors:      <none>
Tolerations:         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                     node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type      Reason      Age   From          Message
  ----      -
Normal    Scheduled   34s   default-scheduler   Successfully assigned default/resourcepod to worker-1
Normal    Pulling     33s   kubelet          Pulling image "nginx"
Normal    Pulled      33s   kubelet          Successfully pulled image "nginx" in 158ms (158ms including waiting). Image size: 62939286 bytes.
Normal    Created     33s   kubelet          Created container: nginx
Normal    Started     32s   kubelet          Started container nginx
root@master:~#

```

```

root@master:~# kubectl apply -f resourcepod.yaml
pod/resourcepod created
root@master:~# kubectl describe pod resourcepod
Name:         resourcepod
Namespace:    default
Priority:      0
Service Account: default
Node:         worker-1/172.31.46.47
Start Time:   Tue, 17 Feb 2026 12:19:30 +0000
Labels:       <none>
Annotations:  cnf.projectcalico.org/containerID: 9f0dd5112f30639b338af7f6fdeef63b9dfa08ae242e01332ce9678100b3e77a
              cnf.projectcalico.org/podIP: 10.10.226.77/32
              cnf.projectcalico.org/podIPs: 10.10.226.77/32
Status:       Running
IP:           10.10.226.77
IPs:          IP: 10.10.226.77
Containers:
  nginx:
    Container ID:  containerd://6c69e18994a32d55d544ffcb09685bf4c3e7556634d91179085ce8a7a4a7ab8c
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
    Port:          <none>
    Host Port:     <none>
    State:         Running
      Started:     Tue, 17 Feb 2026 12:19:32 +0000
    Ready:         True
    Restart Count: 0
    Limits:
      cpu:         200m
      memory:      256Mi
    Requests:
      cpu:         100m
      memory:      128Mi
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-flfk1 (ro)
Conditions:
  Type                               Status
PodReadyToStartContainers            True

```

✓ Task 10 completed.